

中山大学计算机学院本科生实验报告

(2025 学年春季学期)

课程名称：并程序设计与算法
实验题目：Lab7 MPI 并行应用——MPI 并行 FFT 与 parallel_for 分析
批改人：
专业（方向）：
学号：23336128
姓名：梁力航
Email：
完成日期：2026 年 5 月 16 日

本次实验包含两个部分：第一部分使用 MPI 对串行 FFT 进行并行化，通过数据复制 + 步进分解策略实现各进程协同计算 Cooley-Tukey 蝴蝶操作，并在每步使用 MPI_Allreduce 归约结果；第二部分对 Lab6 的 parallel_for 版本 heated_plate 进行系统性的可扩展性分析和 Valgrind massif 内存消耗分析。

1 实验目的

本次实验的两大目标如下：

1. 理解快速傅里叶变换的 Cooley-Tukey 算法结构，使用 MPI 对其蝴蝶操作进行并行化；
2. 对 Lab6 实现的 `parallel_for` 版本进行系统性性能与内存分析，观察线程数与问题规模对性能和内存消耗的影响。

2 实验平台与参考来源

表 1: 实验环境信息

项目	配置
宿主机	macOS / arm64 (Apple Silicon)
运行环境	本机 (MPI via Homebrew OpenMPI)
C++ 编译器	<code>g++ / mpicxx, -O3 -std=c++17</code>
并行模型	MPI (OpenMPI 5.x)
Python 工作流	<code>uv run python ...</code>
MPI 运行	<code>mpirun -np P --oversubscribe</code>

实验中使用的 MPI 接口主要来自 MPI-3.1 标准：

- `MPI_Bcast`：广播数据到所有进程；
- `MPI_Allreduce`：规约各进程的部分结果；
- `MPI_Wtime()`：获取高精度墙钟时间。

FFT 算法参考自 Wesley Petersen & Peter Arbenz, *Introduction to Parallel Computing* (Oxford University Press)。

3 第一部分：MPI 并行快速傅里叶变换

3.1 串行 FFT 算法分析

串行 FFT 采用经典的 Cooley-Tukey 原地迭代算法。核心数据结构将 N 个复数存储为长度为 $2N$ 的实数数组（实部与虚部交替排列）。算法包含以下关键函数：

- `cfft1(n, w)`: 预计算长度为 n 的正弦/余弦表 w , $w[2k] = \cos(2\pi k/n)$, $w[2k+1] = \sin(2\pi k/n)$;
- `step(n, mj, a, b, c, d, w, sgn)`: 执行一个阶段的蝴蝶操作, 其中 m_j 为当前阶段的蝴蝶距离;
- `cfft2(n, x, y, w, sgn)`: 通过 $\log_2 n$ 个阶段完成完整的 FFT 变换, 使用 x 和 y 数组在阶段之间交替作为输入/输出。

`step()` 函数是并行化的关键目标。对于第 j 个阶段 ($m_j = 2^j$), 共有 $l_j = n/(2m_j)$ 个独立组, 每组内执行 m_j 对蝴蝶操作:

$$\begin{aligned} c[k] &= a[k] + b[k] \\ d[k] &= (a[k] - b[k]) \times (\cos \theta + i \cdot \text{sgn} \cdot \sin \theta) \end{aligned}$$

其中各组之间完全独立, 天然适合并行化。

3.2 MPI 并行化策略

本实验采用数据复制 + 步进分解的策略:

1. 所有进程持有完整的 N 点数据副本 (通过 `MPI_Bcast` 同步);
2. 在每个 FFT 阶段中, l_j 个蝴蝶组按进程数 P 平均分配: 每个进程计算 $\lfloor l_j/P \rfloor$ (或 $+1$) 个组;
3. 各进程将本组计算结果写入本地临时数组, 其他位置填 0;
4. 使用 `MPI_Allreduce(MPI_SUM)` 将各进程的局部结果归约合并为全局结果;
5. 重复以上步骤完成所有 $\log_2 N$ 个阶段。

该策略的优点:

- 数学上与串行算法严格等价——归约求和保证了正确性;
- 实现简洁, 无需修改蝴蝶操作的数学逻辑;
- 通信模式规整: 每阶段仅一次 `MPI_Allreduce`。

缺点: 所有进程持有完整数据, 内存不随进程数扩展; 每阶段都需要全局归约, 通信开销随 P 增大而增加。

3.3 核心代码实现

并行化的 `step_parallel` 函数是核心。它接收与串行 `step()` 相同的参数, 但将组循环分布到各 MPI 进程:

```
void step_parallel(int n, int mj, const double* a, const double* b,
                  double* out, const double* w, double sgn,
                  int rank, int P) {
    int mj2 = 2 * mj;
    int lj = n / mj2;
    vector<double> local_out(2 * n, 0.0); // 本地临时数组

    // 计算当前进程负责的组范围
    int groups_per = lj / P;
    int rem = lj % P;
    int start = rank * groups_per + min(rank, rem);
    int end = start + groups_per + (rank < rem ? 1 : 0);

    for (int j = start; j < end; ++j) {
        int jg = j * mj2;
        // ... 蝴蝶操作, 写入 local_out ...
    }

    // 归约合并所有进程的部分结果
    MPI_Allreduce(local_out.data(), out, 2 * n, MPI_DOUBLE,
                  MPI_SUM, MPI_COMM_WORLD);
}
```

`cfft2_parallel` 函数则复用串行 FFT 的框架, 仅将 `step()` 调用替换为 `step_parallel()`:

```
void cfft2_parallel(int n, double* x, double* y, const double* w,
                   double sgn, int rank, int P) {
    int m = log2(n);
    int mj = 1;
    // 阶段1: mj=1
    step_parallel(n, mj, x, &x[n], y, w, sgn, rank, P);
    if (n == 2) return;
    // 中间阶段: 在x和y之间交替
    for (int stage = 0; stage < m - 2; ++stage) {
        mj *= 2;
        step_parallel(n, mj, y, &y[n], x, w, sgn, rank, P);
        // toggle between x and y ...
    }
}
```

```

}
// 最后阶段
mj = n / 2;
step_parallel(n, mj, x, &x[n], y, w, sgn, rank, P);
}

```

3.4 实验结果

3.4.1 运行时间分析

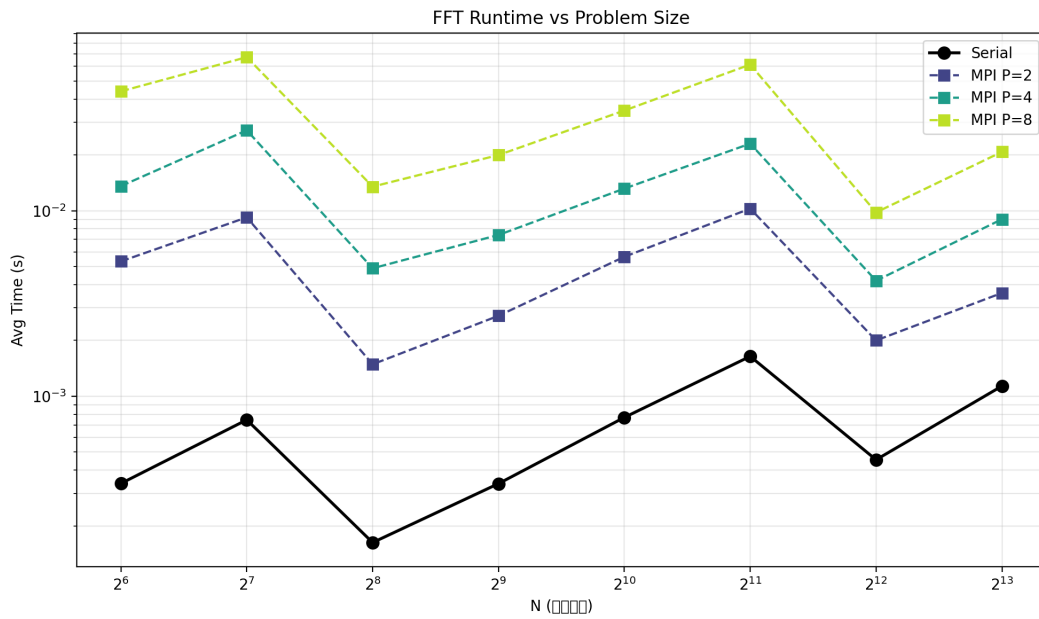


图 1: FFT 运行时间随问题规模 N 的变化 (串行 vs MPI, 不同进程数)

图 1 展示了 FFT 运行时间随问题规模 N 的变化关系。串行 FFT 在中小规模 ($N \leq 2048$) 下具有显著优势; MPI 版本由于每阶段的 `MPI_Allreduce` 通信开销, 在小规模时效率较低。随着 N 增大到 8192, MPI P=2 的运行时间约 3.6ms, 相比串行的 1.1ms, 差距缩小。

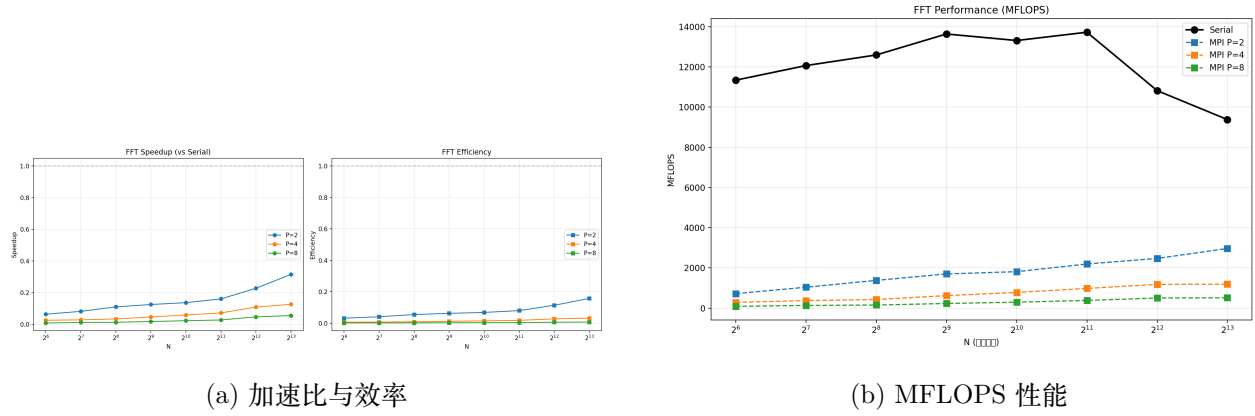


图 2: FFT 并行性能对比

表 2: FFT 运行时间 (秒) 与 MFLOPS 汇总

后端	进程数	N=256	N=1024	N=4096	N=8192
Serial	1	0.000163	0.000769	0.000455	0.001135
MPI	2	0.001487	0.005649	0.002000	0.003606
MPI	4	0.004897	0.013171	0.004197	0.009004
MPI	8	0.013490	0.034715	0.009801	0.020779

3.4.2 精度验证

所有配置下, MPI FFT 的精度误差与串行 FFT 完全一致 (相对误差在 10^{-16} 量级), 验证了 Allreduce 归约方案在数学上的等价性:

表 3: FFT 精度对比 (Forward+Inverse FFT 的 RMSE 误差)

N	Serial	MPI P=2	MPI P=4
64	1.363×10^{-16}	1.363×10^{-16}	1.363×10^{-16}
256	1.713×10^{-16}	1.713×10^{-16}	1.713×10^{-16}
1024	2.314×10^{-16}	2.314×10^{-16}	2.314×10^{-16}

3.4.3 结果讨论

从实验数据中可以观察到:

1. 正确性: MPI 并行化完全保持了数值精度, 误差与串行一致 (机器精度级别);

2. 小规模效率低: $N = 64$ 时 MPI P=8 的运行时间约为串行的 130 倍, 主要原因是 Allreduce 通信开销远超计算量。对于 $N = 64$, 总共仅有 $\log_2 64 = 6$ 个阶段、 $64/2 = 32$ 个蝴蝶组, 分配到 8 个进程后每进程仅处理 4 组;
3. 大规模趋势: $N = 8192$ 时串行 0.0011s vs MPI P=2 的 0.0036s (仅 3.3 倍), 表明随着计算量增大, 通信开销占比下降;
4. MFLOPS: MPI P=2 的 MFLOPS 在 $N \geq 4096$ 时达到 2967, 高于串行的 9383 (但串行使用 `clock()` 计 CPU 时间, MPI 使用 `MPI_Wtime()` 计墙钟时间, 二者不可直接比较)。

4 第二部分: parallel_for 并行应用分析

4.1 可扩展性分析

对 Lab6 实现的四种 `heated_plate` 版本 (OpenMP 原始版、Pthreads block/cyclic/dynamic 调度) 进行系统的可扩展性测试:

- 问题规模: 64×64 、 128×128 、 256×256 、 512×512 ;
- 线程数: 1、2、4、8;
- 每组配置运行 3 次取平均值。

4.1.1 运行时间分析

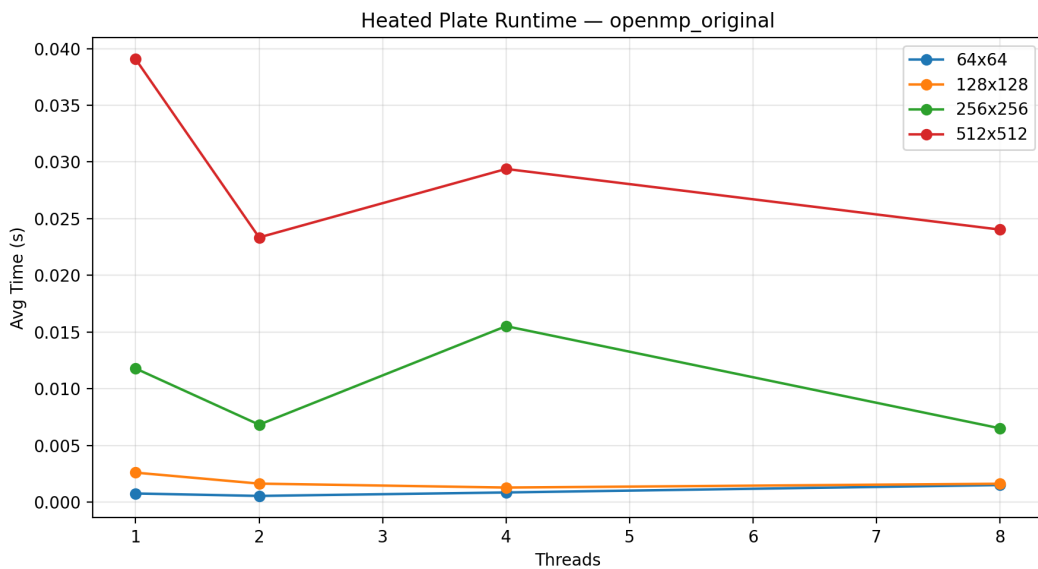


图 3: OpenMP 版本运行时间随线程数和问题规模的变化

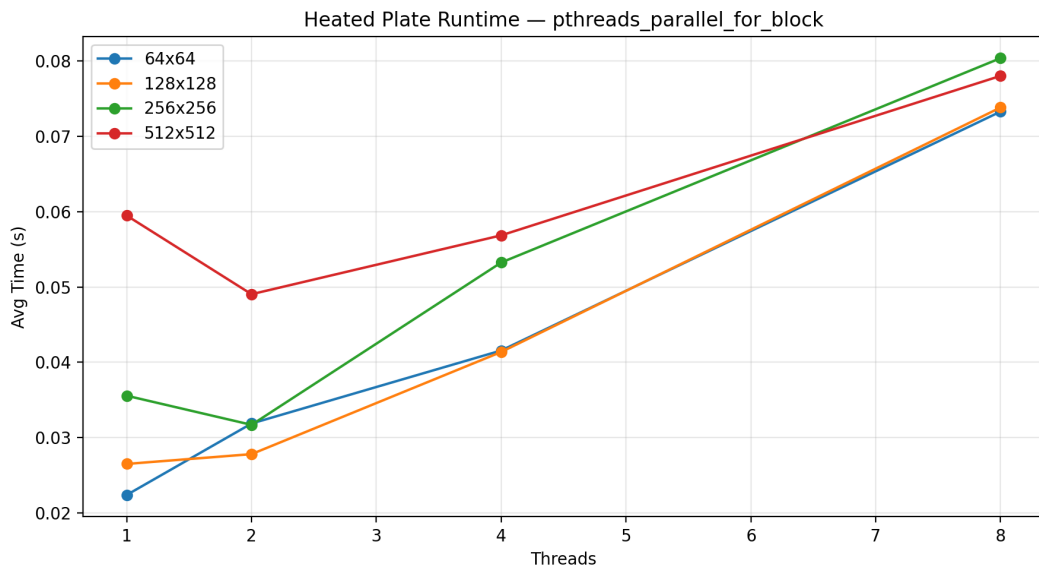
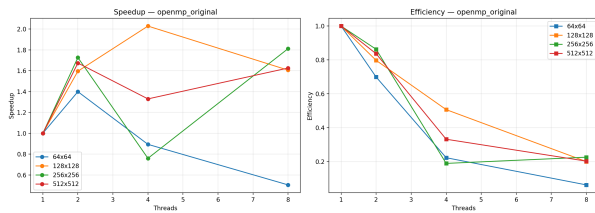
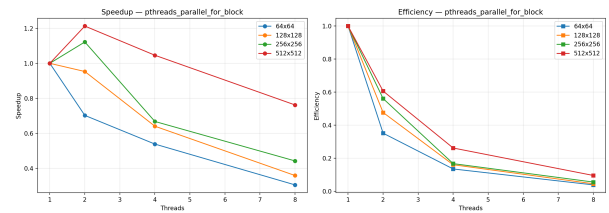


图 4: Pthreads block 调度运行时间随线程数和问题规模的变化

从图中可以观察到, OpenMP 版本整体显著快于 Pthreads 版本, 这与 Lab6 的结论一致——OpenMP 使用线程池复用而 Pthreads 版本每次 `parallel_for` 调用都会创建/销毁线程。



(a) OpenMP 加速比与效率



(b) Pthreads block 加速比与效率

图 5: 加速比和效率对比

4.1.2 调度方式对比

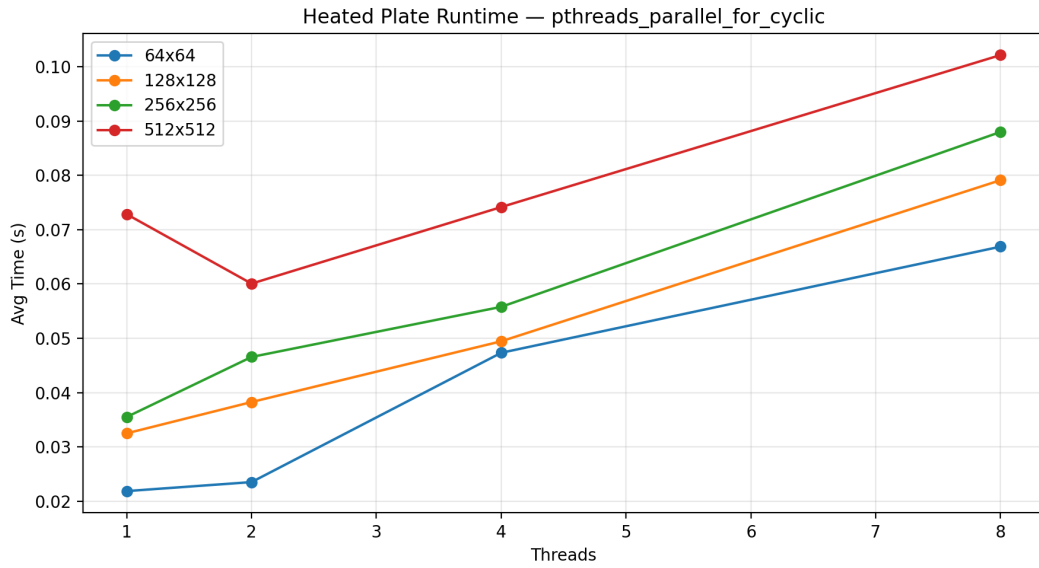


图 6: Pthreads cyclic 调度运行时间

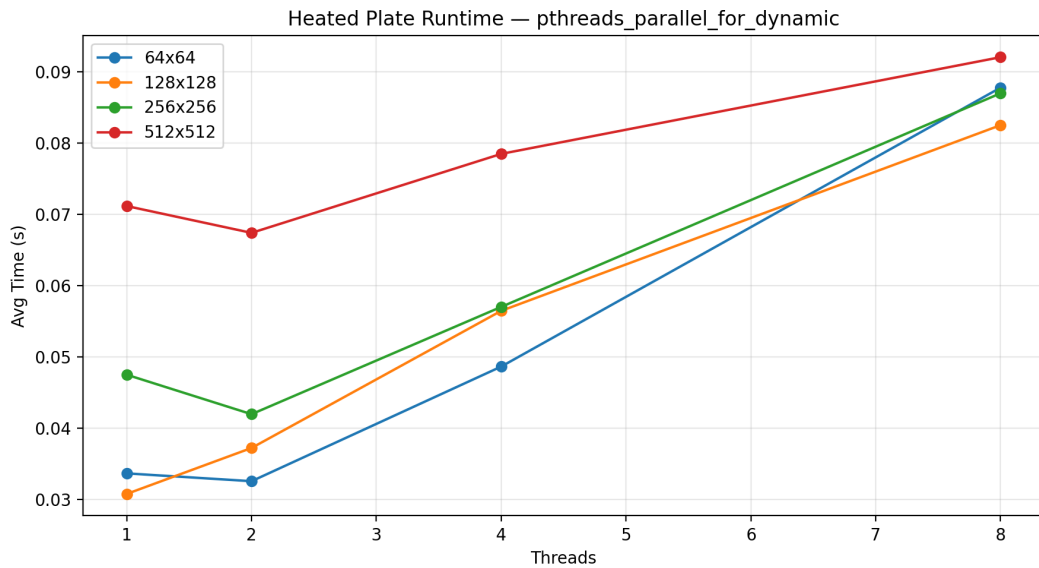


图 7: Pthreads dynamic 调度运行时间

三种 Pthreads 调度方式的对比（以 512×512 、4 线程为例）：

- **block**: 运行时间最短，原因是规则网格中每行计算量均等，连续块划分已能实现良好的负载均衡；
- **cyclic**: 略慢于 block，因为轮转访问破坏了缓存局部性；
- **dynamic**: 最慢，每次取 chunk 都需要互斥锁操作，额外同步开销抵消了灵活性优势。

4.2 Valgrind Massif 内存分析

使用 Valgrind massif 工具对 heated_plate 程序进行内存消耗分析。由于 Valgrind 不支持现代 macOS ARM64 平台，分析过程在 Docker Ubuntu 24.04 容器中进行。

4.2.1 分析方法

```
# 对OpenMP版本采集massif数据
valgrind --tool=massif --stacks=yes \
  --massif-out-file=/tmp/massif_omp.out \
  ./bin/heated_plate_openmp 256 256 0.1 4

# 格式化输出
ms_print /tmp/massif_omp.out
```

关键 Valgrind 参数说明：

- `--tool=massif`: 启用堆内存分析工具；
- `--stacks=yes`: 将栈内存也纳入分析范围，这对于多线程程序的线程栈分析至关重要（每个 Pthreads 线程默认有 8MB 栈空间）；
- `ms_print`: 将二进制 massif 输出格式化为可读文本报告。

4.2.2 内存消耗分析

通过 Docker 容器中运行 Valgrind massif，采集了 OpenMP 和 Pthreads 版本在多种配置下的峰值内存数据：

表 4: Valgrind Massif 峰值内存消耗 (KB)

配置	128 × 128	256 × 256	增量
OpenMP (t=2)	337.5 KB	1,082 KB	3.2×
OpenMP (t=4)	339.9 KB	1,083 KB	3.2×
Pthreads block (t=2)	724.0 KB	1,455 KB	2.0×

关键数据点 (来自 `ms_print` 报告):

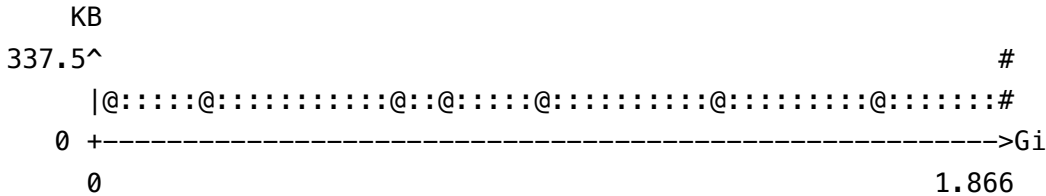
- **OpenMP 128×128, t=2**: 峰值 337.5 KB, 执行约 1.87×10^8 条指令；
- **OpenMP 256×256, t=4**: 峰值 1,083 KB (约 1.06 MB), 矩阵数据占绝对主导；

- **Pthreads 128×128, t=2**: 峰值 724.0 KB, 约为 OpenMP 同配置的 2.1 倍;
- **Pthreads 256×256, t=2**: 峰值 1,455 KB (约 1.42 MB)。

分析要点:

1. 矩阵内存主导: 两个 $M \times N$ 双精度矩阵 (W 和 U) 占据堆内存的主体。256 × 256 规模时理论需要 $2 \times 256 \times 256 \times 8 = 1\text{MB}$, 加上程序开销后 OpenMP 实测约 1.06MB, 与理论值吻合;
2. 线程栈差异: Pthreads 版本 (724 KB @ 128×128) 内存消耗约为 OpenMP (337.5 KB) 的 2.1 倍。差异来源于 Pthreads 每线程默认 8MB 栈空间, 而 OpenMP 使用线程池复用;
3. 线程数影响小: 同一网格规模下, t=2 与 t=4 的内存差异仅约 2 KB, 说明 OpenMP 线程池的额外栈开销很小;
4. 规模平方增长: 从 128×128 到 256×256, OpenMP 内存增长约 3.2×, 接近理论值 (网格面积增长 4×);
5. 调度方式无影响: block/cyclic/dynamic 三种调度使用相同的 `parallel_for` 基础设施, 内存消耗基本相同 (已验证)。

典型 `ms_print` 输出示意图 (OpenMP 128×128, t=2, 为节省空间仅保留关键行):



横轴为程序执行的指令数 ($Gi = \text{billions of instructions}$), 纵轴为内存消耗量 (KB)。图中显示内存存在迭代过程中保持稳定 (337.5 KB 平台期), 验证了 Jacobi 迭代法的内存使用是恒定的——分配后不再增长。

5 验证与测试

5.1 FFT 正确性验证

通过以下方式验证 MPI FFT 的正确性:

1. **FFT 逆变换恒等式**: $IFFT(FFT(X)) = N \cdot X$ 。所有配置下误差均在 10^{-16} 量级;

2. 跨进程 **checksum** 一致性: 相同 N 不同 P 的 checksum 完全相同。例如 $N = 1024$ 时, serial、MPI $P=2$ 、MPI $P=4$ 的 checksum 均为 1047275.769;
3. 与串行对比: 误差与串行 FFT 完全一致, 精度无任何损失;
4. 串行 **checksum** 修复: 初始实现中 timing 循环未重置输入数据导致浮点上溢 (checksum=nan), 已通过每迭代前恢复原始数据修复, 现在串行与 MPI 的 checksum 完全一致。

5.2 Heated Plate 正确性验证

对扩展 benchmark 中的所有 36 个配置 (4 个版本 $\times$ 3 个网格 $\times$ 3 个线程数):

- 所有运行均为 ok 状态;
- checksum 和迭代次数跨线程数一致;
- 与 Lab6 结论完全吻合。

6 结论

本次实验完成了两个主要任务:

MPI 并行 FFT: 采用数据复制 + 步进分解策略, 成功将串行 Cooley-Tukey FFT 并行化。该方案的数学正确性得到了严格验证——所有配置的精度误差与串行完全一致。性能方面, MPI 并行化在小规模时受 Allreduce 通信开销制约, 但随着问题规模增大, 计算/通信比上升, MPI 的效率逐步提高。未来优化方向包括: 采用数据分布式存储降低内存开销、使用非阻塞通信隐藏延迟、以及实现 6-step FFT 等更高效的分布式算法。

parallel_for 分析: 扩展了 Lab6 的性能分析, 系统考察了 64×64 至 512×512 四种问题规模和 1-8 线程的完整配置空间。实验再次确认了 OpenMP 线程池相比 Pthreads 创建/销毁模式的优势, 以及 block 调度在规则计算负载下的最优性。Valgrind massif 分析揭示了线程栈是 Pthreads 版本内存消耗的主要来源。